JavaScript for Playwright

Student Notes — Simple Definitions, Syntax & Playwright Usage

1. JavaScript Basics

JavaScript

Definition: JavaScript is a programming language used to add logic, behaviour, and functionality to applications.

Why JavaScript is used in Playwright

JavaScript is used to write Playwright test cases and handle test data, conditions, loops, functions and browser actions.

console.log()

Definition: Used to display information in the console.

```
console.log(value);

console.log("Login successful");
```

2. Variables & Data Types

Variable

Definition: A variable is used to store and reuse data.

let

Definition: Declares a variable whose value can be changed, but it cannot be redeclared in the same scope.

const

Definition: Declares a variable whose value cannot be reassigned or redeclared in the same scope.

var

Definition: Declares a variable that can be reassigned and redeclared in the same scope.

Playwright Usage

- Store usernames and passwords
- Store product names and prices
- Store locators and test results

```
const username = "admin";
const password = "admin123";
```

Data Types

- **Number** — numeric values such as price and quantity.
- **String** — text values such as username and product name.
- **Boolean** — true or false values.
- **Undefined** — a declared variable without an assigned value.
- **Null** — represents an intentionally empty value.
- **Object** — related data stored as key-value pairs.
- **Array** — multiple values stored in one variable.

```
const price = 500; // Number
const username = "admin"; // String
const isLoggedIn = true; // Boolean
```

```
const products = ["Laptop", "Phone"]; // Array
```

3. Operators, Conditions & Loops

Operators

Definition: Operators are symbols used to perform operations on values.

Arithmetic Operators

+ – * / %

Used for calculations such as price, quantity and total amount.

Assignment Operators

= += –=

Used to assign or update values.

Comparison Operators

== === != !== > < >= <=

Used to compare expected and actual values.

Logical Operators

&& || !

Used to combine conditions.

if

Definition: Executes code when a condition is true.

```
if (condition) {
// code
}
```

if-else

Definition: Executes one block when true and another block when false.

```
if (condition) {
// true
} else {
// false
}
```

else if

Definition: Used when multiple conditions need to be checked.

```
if (condition1) {

} else if (condition2) {

} else {

}
```

Loops

Definition: Loops repeat the same block of code multiple times.

for loop

```
for (initialization; condition; increment) {
// code
}
```

while loop

```
while (condition) {
// code
}
```

Playwright Usage

- Iterate through multiple products
- Read multiple table rows
- Work with multiple elements
- Run the same action for multiple test data values

4. Functions

Function

Definition: A function is a reusable block of code designed to perform a specific task.

```
function functionName(parameters) {
// code
}
```

Calling a Function

Definition: Calling a function executes the function.

```
functionName(arguments);
```

Parameter

Definition: A variable defined in the function to receive a value.

Argument

Definition: The actual value passed to a function.

return

Definition: Sends a value back from a function.

```
return value;
```

Playwright Usage

- login()
- logout()
- searchProduct()
- addToCart()

5. Arrays & Objects

Array

Definition: An array stores multiple values in a single variable.

```
const products = ["Laptop", "Phone", "Tablet"];
```

Index

Definition: The position of an element. JavaScript arrays start from index 0.

Example: Laptop → 0, Phone → 1, Tablet → 2

Common Array Methods

- **push()** — adds an element at the end.
- **pop()** — removes the last element.
- **shift()** — removes the first element.
- **unshift()** — adds an element at the beginning.
- **slice()** — returns part of an array without changing the original.
- **splice()** — adds, removes or replaces elements and changes the original.
- **join()** — combines array elements into a string.
- **sort()** — sorts array elements.

Object

Definition: An object stores related data as key-value pairs.

```
const user = {
username: "admin",
password: "admin123"
};
```

Accessing Object Properties

```
user.username;
user["username"];
```

Playwright Usage

Objects are useful for storing structured test data such as login details, product details and customer details.

6. OOPS — Required Basics

OOPS

Definition: Object-Oriented Programming is an approach that organizes code using classes and objects.

Class

Definition: A class is a blueprint used to create objects.

```
class LoginPage {
// methods
}
```

Object

Definition: An object is an instance created from a class.

```
const loginPage = new LoginPage();
```

Encapsulation

Definition: Encapsulation keeps related data and methods together inside a class.

Inheritance

Definition: Inheritance allows one class to use properties and methods of another class.

```
class Child extends Parent {
// code
}
```

Polymorphism

Definition: Polymorphism allows the same method name to have different behaviour.

Playwright Usage

Page Object Model uses classes and objects to organize reusable page actions and locators.

7. Looping & Array Methods

forEach()

Definition: Executes a callback function for each element in an array.

```
array.forEach((element) => {
// code
});
```

for...of

Definition: Iterates through the values of an array or other iterable.

```
for (let value of array) {
// code
}
```

for...in

Definition: Mainly iterates through the keys of an object.

```
for (let key in object) {
// code
}
```

Quick Difference

Method	Main Use	Gets
forEach()	Array iteration	Values
for...of	Array / iterable	Values
for...in	Object iteration	Keys

map()

Definition: Creates a new array by transforming each element.

```
array.map((element) => {
return newValue;
});
```

filter()

Definition: Creates a new array containing elements that satisfy a condition.

```
array.filter((element) => {
return condition;
});
```

reduce()

Definition: Reduces an array into one accumulated result.

```
array.reduce((accumulator, currentValue) => {
return result;
}, initialValue);
```

Playwright Usage

- map() — transform or extract test data
- filter() — select required data
- reduce() — calculate totals such as price or quantity

8. Promises, async/await & Execution

Asynchronous Programming

Definition: Allows JavaScript to handle tasks that take time, such as network or browser operations.

Promise

Definition: A Promise represents the future result of an asynchronous operation.

Promise States

- **Pending** — operation is still running.
- **Fulfilled** — operation completed successfully.
- **Rejected** — operation failed.

resolve()

Indicates successful completion of a Promise.

reject()

Indicates failed completion of a Promise.

.then()

Handles the successful result of a Promise.

.catch()

Handles a rejected Promise.

async

Definition: Makes a function asynchronous and causes it to return a Promise.

```
async function functionName() {
// code
}
```

await

Definition: Waits for a Promise result inside an async function.

```
const result = await promise;
```

Playwright Usage

Playwright actions commonly use **async/await** because browser operations are asynchronous.

```
await page.goto("https://example.com");
await page.locator("#username").fill("admin");
await page.locator("#login").click();
```

GEC — Global Execution Context

Definition: The execution context created when JavaScript starts executing global code.

Call Stack

Definition: A mechanism that keeps track of functions currently being executed.

Quick Revision — JavaScript for Playwright

- Variables → store test data and locators
- Data Types → handle different kinds of test data
- Operators → calculations and comparisons
- Conditions → conditional test actions
- Loops → work with multiple elements/data
- Functions → create reusable actions

• Arrays & Objects → organize test data

• Array Methods → process test data

• OOPS → understand Page Object Model structure

• Promises & async/await → handle asynchronous Playwright actions

Important for Playwright: Variables, conditions, loops, functions, arrays, objects and especially **async/await** are used frequently while writing automation tests.